

CHƯƠNG 2

MAPREDUCE VÀ HADOOP CƠ BẢN

Dữ liệu lớn - INT4418

Giảng viên: TS. Nguyễn Kiều Linh

Khoa Trí tuệ nhân tạo

Học viện Công nghệ Bưu chính Viễn thông



Hà Nội, năm 2026

Mục tiêu

Mục tiêu kiến thức

- * Mô hình lập trình hàm
- * Key - value: Mapper, Reducer, shuffle và sort
- * Combiner, Partitioner và data skew
- * HDFS, data locality và cụm Hadoop

Mục tiêu vận dụng

- * Thiết kế một MapReduce job cho dữ liệu toán
- * Nhận biết bài toán phù hợp/không phù hợp
- * Gắn khái niệm vào project AI4Math
- * Đề xuất thí nghiệm scalability có kiểm soát

Bài toán trung tâm

Giả sử ta có 1 TB theorem, premise và proof trace từ nhiều nguồn AI4Math.

Bộ nhớ

Không thể nạp toàn bộ dữ liệu vào RAM

Thời gian

Chạy tuần tự mất hàng giờ hoặc hàng ngày

Độ tin cậy

Một tiến trình lỗi có thể làm mất cả job

Tăng trưởng

Dữ liệu và số truy vấn tiếp tục tăng

Câu hỏi thiết kế: chia dữ liệu ở đâu, đưa tính toán tới dữ liệu thế nào, gom kết quả ra sao?

Liên hệ với tám project AI4Math

1. Math Data Lake

HDFS, block, replication, partition

2. Data Quality

Map/Reduce, Combiner, skew

3. Sparse Retrieval

Inverted index, postings, partitioner

4. Dense/ANN

Shard, local top-k, merge kết quả

5. Autoformalization

Queue, worker, retry, idempotence

6. Proof Repair

Vòng lặp, checkpoint; không thuần MapReduce

7. Proof Analytics

Event aggregation, Parquet, skew

8. Math Tutor

Batch đối chiếu streaming

Không ép công cụ: project phải chứng minh quyết định kiến trúc và khả năng mở rộng.

2.2 Nguyên lý lập trình hàm

MapReduce dễ phân tán vì công việc được mô tả bằng các hàm trên dữ liệu.

Hàm Map

- * Nhận một bản ghi đầu vào
- * Không cần biết bản ghi khác
- * Phát ra 0, 1 hoặc nhiều cặp key–value
- * Dễ chạy song song trên nhiều partition

Hàm Reduce

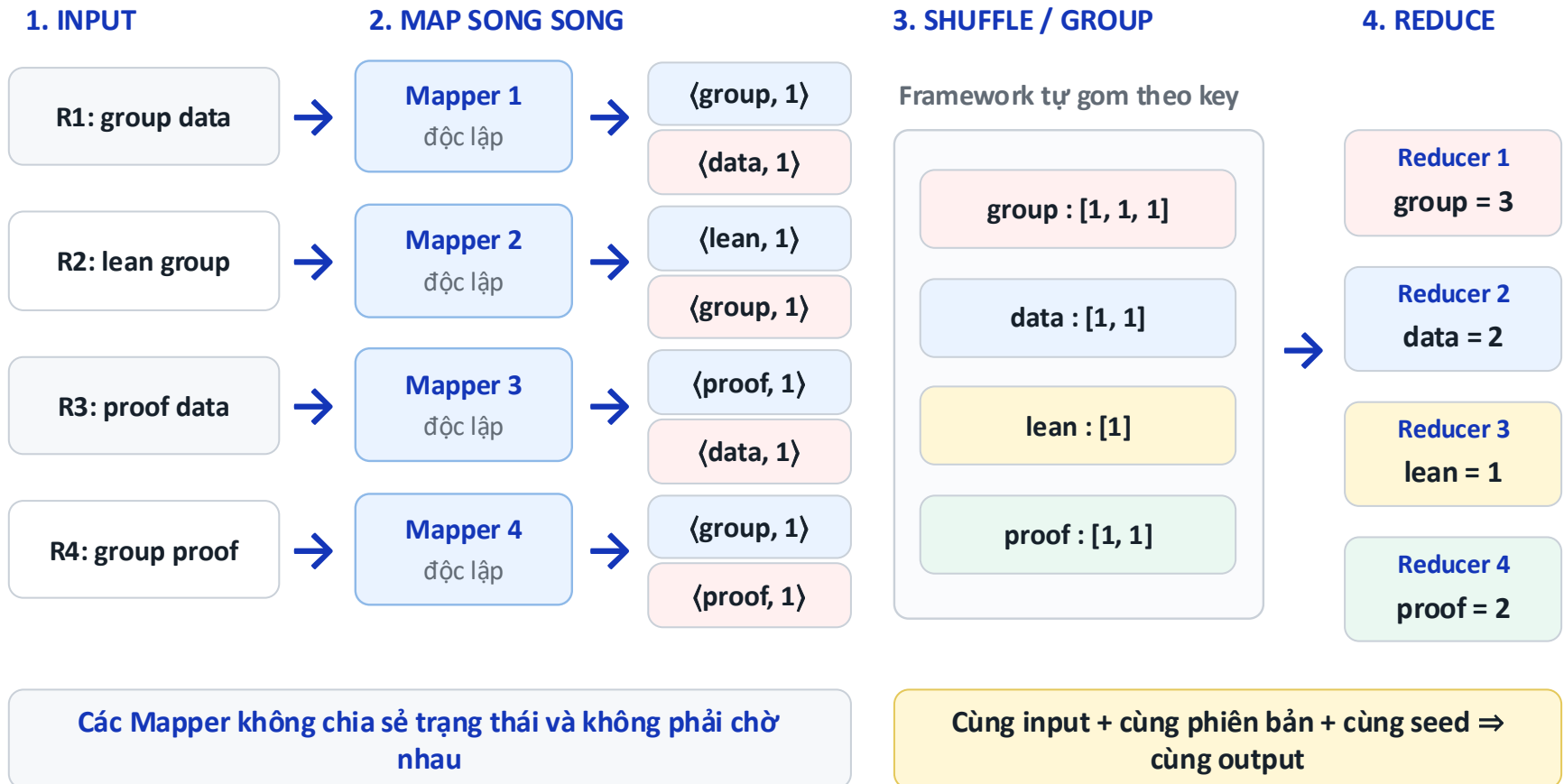
- * Nhận một key và danh sách value
- * Tổng hợp độc lập theo key
- * Phát ra kết quả cuối hoặc trung gian
- * Tránh phụ thuộc trạng thái toàn cục

TƯ DUY CỐT LỖI

same input → same output; không sửa dữ liệu dùng chung; chia nhỏ và ghép kết quả bằng quy tắc rõ ràng.

Map và Reduce: chia việc độc lập, gom theo key

Mỗi Mapper xử lý một bản ghi riêng. Framework gom các value có cùng key trước khi Reduce tổng hợp.



Nếu Mapper phụ thuộc trạng thái bên ngoài đang thay đổi, việc chạy lại sau lỗi có thể tạo kết quả khác.

Mô hình key-value: ngôn ngữ chung của pipeline

Mỗi bước cần xác định rõ kiểu của khóa và giá trị.

Đầu vào

$\langle k_1, v_1 \rangle$

$\langle \text{file_offset}, \text{theorem_record} \rangle$

Map phát ra

$\langle k_2, v_2 \rangle$

$\langle \text{token}, \text{theorem_id} \rangle$

Reduce nhận

$\langle k_2, \text{list}(v_2) \rangle$

$\langle \text{token}, \text{list}(\text{theorem_id}) \rangle$

Đầu ra

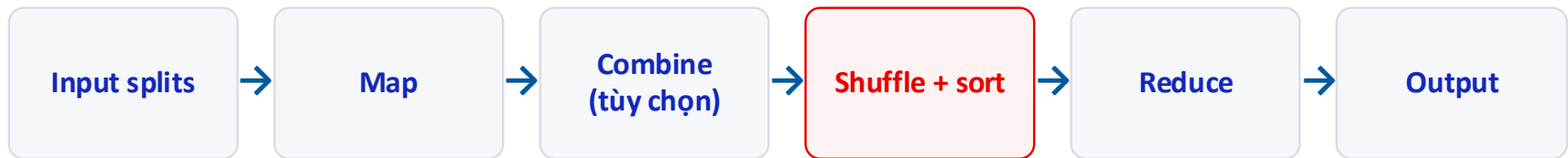
$\langle k_3, v_3 \rangle$

$\langle \text{token}, \text{posting_list} \rangle$

Data contract: kiểu dữ liệu • ý nghĩa khóa • schema của value • quy tắc xử lý lỗi

2.3 Luồng MapReduce hoàn chỉnh

Framework điều phối phần việc mà lập trình viên không nên tự viết lại.



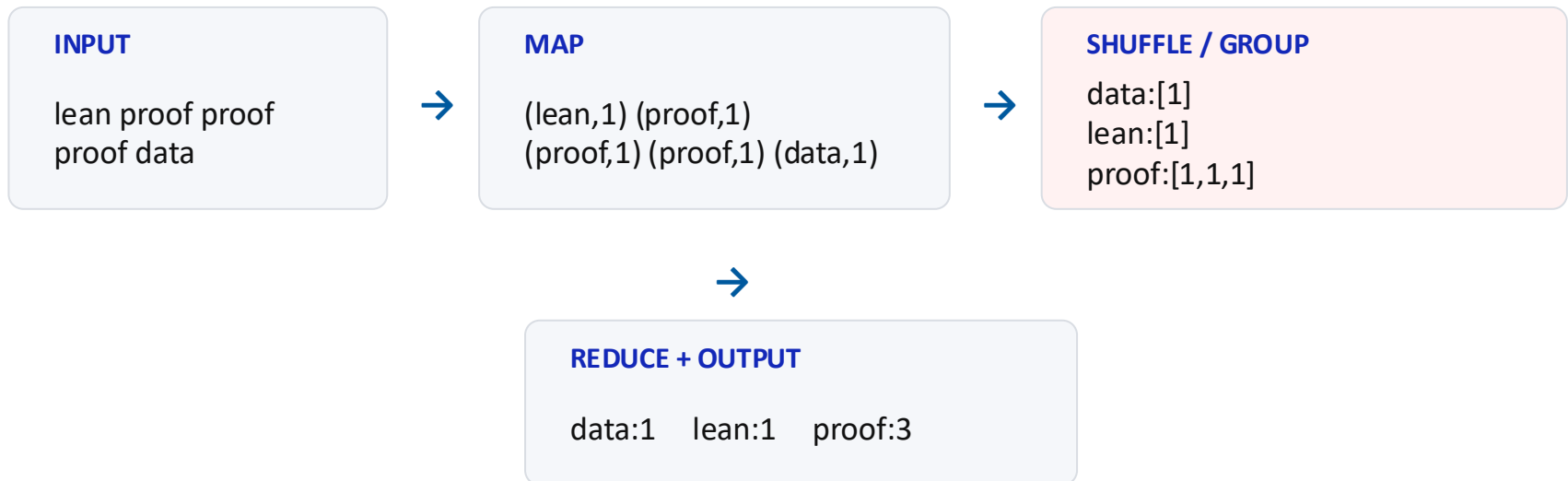
- * Split: chia đầu vào thành các phần độc lập
- * Shuffle: chuyển dữ liệu để cùng key về cùng reducer
- * Sort/group: sắp và gom các value theo key
- * Fault recovery: theo dõi và chạy lại task lỗi

CHI PHÍ THƯỜNG BỊ QUÊN

Đọc/ghi đĩa + tuần tự hóa + truyền mạng + mất cân bằng tải.

Ví dụ: WordCount để đọc đúng luồng dữ liệu

WordCount đơn giản, nhưng cho thấy rõ Map → shuffle → Reduce.

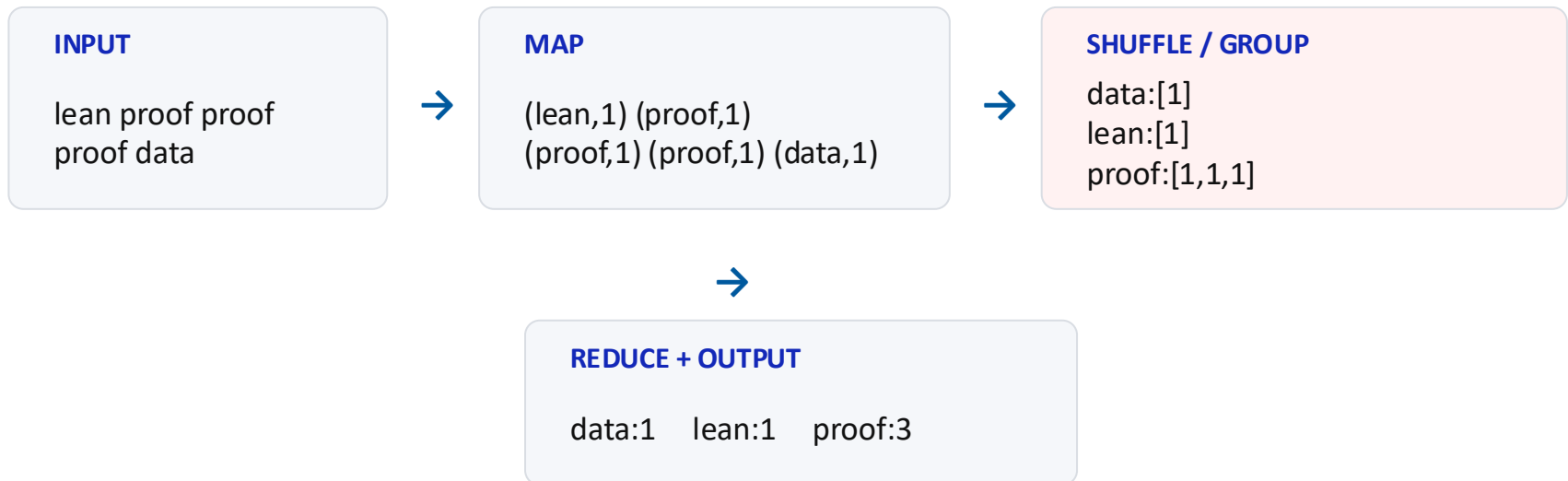


Câu hỏi:

- * Máy nào quyết định ba cặp proof về cùng một reducer?
- * Reducer có biết proof nằm ở dòng nào không?
- * Nếu từ proof xuất hiện một tỷ lần thì có vấn đề gì không?

Ví dụ: WordCount để đọc đúng luồng dữ liệu

WordCount đơn giản, nhưng cho thấy rõ Map → shuffle → Reduce.



Điều rút ra từ ví dụ này:

- * Key quyết định cách gom
- * Map có thể phát nhiều cặp
- * Reduce chỉ thấy một key mỗi lượt

Điều chưa rút ra từ ví dụ này:

- * Skew và key rất lớn
- * Schema phức tạp, lỗi dữ liệu
- * Chi phí index và truy vấn thật

Ví dụ AI4Math: xây chỉ mục premise retrieval

Mục tiêu: từ truy vấn theorem, tìm nhanh các premise có khả năng liên quan.

BẢN GHI THEOREM

id • statement tự nhiên • biểu thức Lean •
namespace • dependencies

CHỈ MỤC ĐẢO

token / symbol → danh sách theorem_id và
trọng số

Ví dụ

theorem T42: `Nat.add_comm a b`

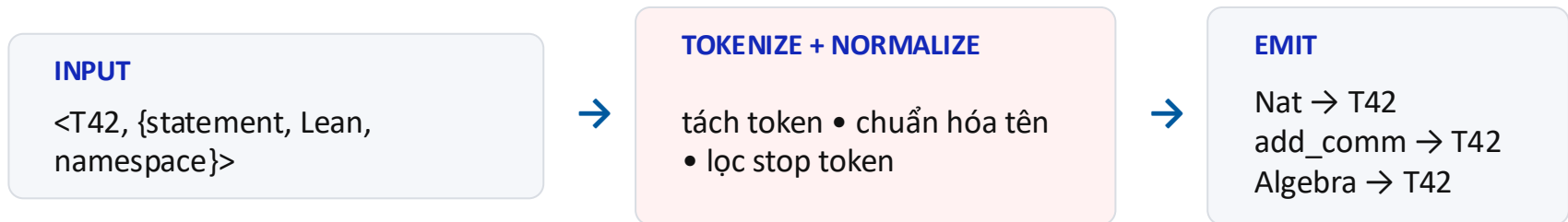
`Nat` → [T3, T17, T42, ...]

`add_comm` → [T42, T81, ...]

- * Index build là batch job: xử lý theo lô khi thư viện thay đổi
- * Query serving là online service: trả lời truy vấn với độ trễ thấp
- * Cần tách rõ đường xây chỉ mục và đường phục vụ truy vấn

Mapper cho premise retrieval

Mapper biến một theorem thành nhiều tín hiệu truy hồi độc lập.



Quyết định sử dụng

- * Token là từ tự nhiên, ký hiệu Lean hay cả hai?
- * Value chỉ có theorem_id hay thêm term frequency/field?
- * Bản ghi lỗi được bỏ, sửa hay chuyển sang dead-letter output?
- * Làm sao tránh cùng theorem phát trùng một token quá nhiều lần?

Shuffle và posting list: nơi chi phí bộc lộ

Mọi value có cùng token phải gặp nhau trước khi tạo posting list.

M1: <Nat,T1>

M1: <group,T1>

M2: <Nat,T8>

M3: <Nat,T42>

M3: <add,T42>



SHUFFLE + GROUP

Nat → [T1, T8, T42]
group → [T1]
add → [T42]



POSTING LIST

Nat: 3 docs
[T1, T8, T42]

ĐO LƯỜNG

map output records • shuffle bytes • spill to disk • reducer max/median time

Reducer: tạo chỉ mục có thể truy vấn và tái lập

Reducer không chỉ “gom lại”; nó quyết định cấu trúc và chất lượng đầu ra.

Reducer nhận

- * token
- * danh sách theorem_id
- * tần suất/field/weight tùy thiết kế
- * có thể có phần tử trùng

Reducer thực hiện

- * loại trùng hoặc cộng tần suất
- * sắp xếp ổn định
- * nén posting list
- * ghi thống kê và version metadata

OUTPUT CONTRACT

token • document frequency • posting list • schema version • build timestamp/data snapshot

2.4 Combiner: tổng hợp cục bộ, nhưng phải an toàn

Combiner có thể chạy 0, 1 hoặc nhiều lần; kết quả cuối không được phụ thuộc số lần chạy.

An toàn khi có tính chất phù hợp

- * sum: cộng số lượng
- * min / max
- * merge các tập hợp với union
- * partial count: (sum, count)

Không an toàn nếu làm trực tiếp

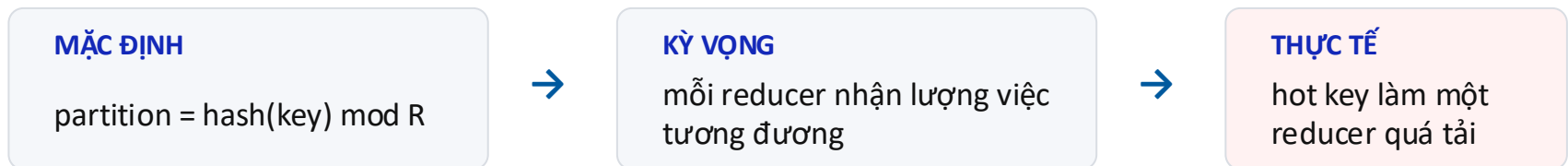
- * average đơn lẻ của từng partition
- * median
- * top-k nếu merge sai quy tắc
- * logic phụ thuộc thứ tự hoặc trạng thái

VÍ DỤ AVERAGE ĐÚNG

Map phát (sum, count) → Combiner cộng từng phần → Reducer cộng toàn cục → $\text{average} = \text{total_sum} / \text{total_count}$

Partitioner và data skew

Partitioner chọn reducer; phân bố key quyết định cân bằng tải.



Dữ liệu toán thường lệ

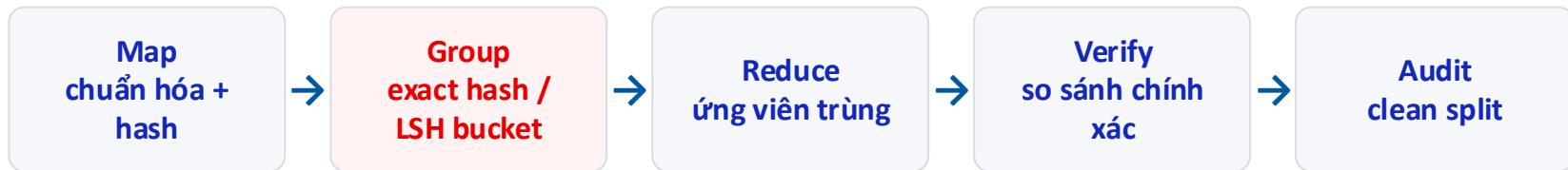
- * Token phổ biến: theorem, Eq, Nat, simp...
- * Một số namespace hoặc tactic chiếm tỷ lệ rất lớn
- * Proof trace dài tạo record nặng hơn nhiều record khác
- * Kích thước value quan trọng, không chỉ số lượng key

GIẢM SKEW

stop token • sampling • salting hot key • two-stage aggregation • custom partitioner

Nhóm 2: Data Quality dưới góc nhìn MapReduce

Mục tiêu: phát hiện exact duplicate, near duplicate và leakage ở quy mô lớn.



Baseline

- * Exact hash sau chuẩn hóa cơ bản
- * So sánh trong phạm vi nhỏ
- * Báo precision/recall trên mẫu gắn nhãn

Scalability

- * 10K → 100K → 1M bản ghi
- * 1 → 2 → 4 worker hoặc partition
- * runtime • shuffle • candidate pairs

RỦI RO

LSH bucket quá lớn tạo bùng nổ số cặp; leakage phải kiểm tra xuyên train/validation/test.

Nhóm 7: Phân tích proof-search theo sự kiện

Mỗi bước tìm kiếm là một event; tổng hợp đúng giúp thấy điểm nghẽn của proof search.

EVENT SCHEMA

theorem_id • state_id • parent • tactic • depth • latency • result • error

Map keys có thể dùng

- * tactic_name
- * error_type
- * depth_bucket
- * theorem_id hoặc model_version

Reduce metrics

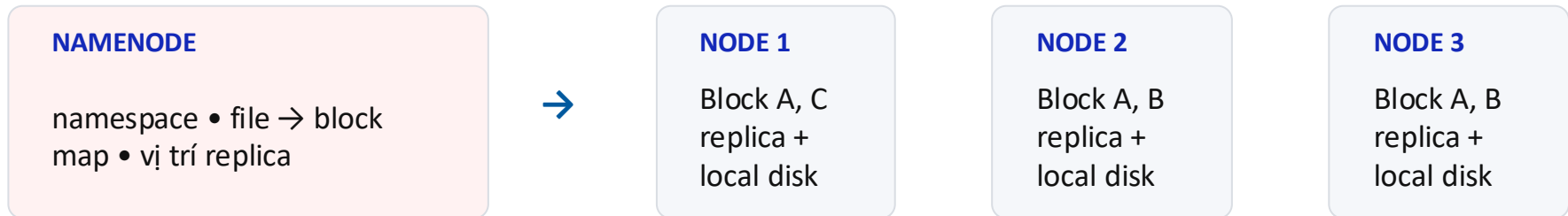
- * count / success rate
- * mean, p50, p95 latency
- * states expanded
- * branching factor / failure profile

THÍ NGHIỆM

Baseline không pruning $\leftrightarrow$ chiến lược pruning; tăng theorem và search budget; báo proof rate cùng chi phí.

2.5 HDFS: hệ thống tệp phân tán cho Math Data Lake

HDFS lưu tệp lớn thành block trên nhiều DataNode và quản lý metadata qua NameNode.



Liên hệ Nhóm 1 - Math Data Lake

- * Lưu raw, cleaned, curated thành các zone rõ ràng
- * Partition theo snapshot/source/split; tránh quá nhiều file nhỏ
- * Parquet cho dữ liệu có schema và truy vấn cột
- * Manifest + provenance + checksum để tái lập

Block, replication và data locality

Ba khái niệm quyết định cách dữ liệu lớn được lưu và xử lý hiệu quả.

BLOCK

Tập lớn → các khối; đơn vị lưu và truyền

Chọn kích thước để giảm metadata và tăng tuần tự

REPLICATION

Nhiều bản sao trên các nút khác nhau

Chịu lỗi nhưng tốn dung lượng và băng thông ghi

DATA LOCALITY

Đưa task tới nơi có block

Giảm truyền dữ liệu lớn qua mạng

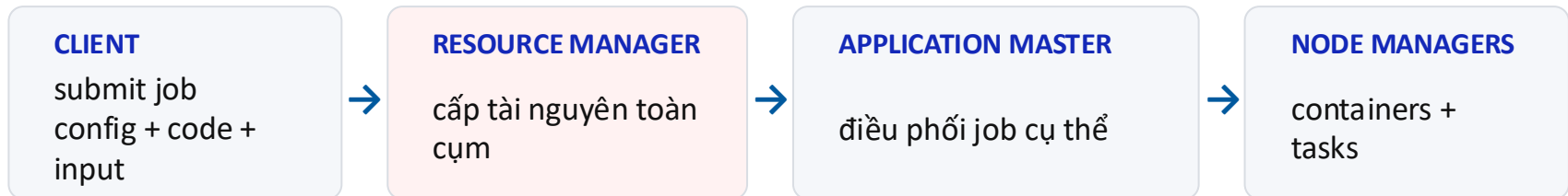
THÍ NGHIỆM NHÓM 1

Cùng dữ liệu: nhiều file nhỏ ↔ file được compact; đo listing time, read throughput, task count và total runtime.

Big Data không chỉ là số byte; layout dữ liệu có thể quyết định chi phí toàn pipeline.

2.6 Kiến trúc cụm Hadoop

Hadoop tách quản lý tài nguyên, thực thi ứng dụng và lưu dữ liệu.



Vòng đời

- * Client nộp job và cấu hình đầu vào/đầu ra
- * ResourceManager cấp container; ApplicationMaster lập lịch task
- * NodeManager báo trạng thái; task lỗi được chạy lại
- * Output được commit khi task/job thành công

AI4MATH WORKERS

Lean/inference workers dùng queue riêng nhưng vẫn cần cấp tài nguyên, heartbeat, retry và idempotence.

Nhóm 4–6: dùng nguyên lý phân tán, không ép Hadoop

Kiến trúc đúng phụ thuộc vào kiểu workload – dạng tải công việc.

N4 Dense/ANN

Shard vector index → local top-k → global merge

Đo Recall@k, latency, memory; cân bằng shard

N5 Autoformalization

Queue problems → inference → Lean verify → store

Đo throughput, compile rate, cost; retry an toàn

N6 Proof repair

Error → classify → propose repair → verify → repeat

Đo success@budget, iterations, token/time cost

QUY TẮC CHỌN

MapReduce hợp batch một chiều; vòng lặp, truy vấn online và ANN thường cần engine/service chuyên biệt.

Nhóm 8: batch hay streaming cho AI Math Tutor?

Cùng một hệ thống có thể cần hai đường xử lý với yêu cầu khác nhau.

Batch - xử lý theo lô

- * Tổng hợp lịch sử theo ngày/tuần
- * Huấn luyện hoặc cập nhật mô hình
- * Báo cáo lớp và cohort
- * Chấp nhận độ trễ phút–giờ

Streaming - xử lý dòng

- * Nhận từng sự kiện lời giải
- * Cập nhật learner state gần thời gian thực
- * Phát hiện bế tắc và tạo hint
- * Yêu cầu độ trễ giây hoặc thấp hơn

KIẾN TRÚC GỢI Ý

event log → stream processor → learner state/hint; đồng thời đẩy vào lake để batch analytics và tái huấn luyện.

Yêu cầu cho mỗi nhóm tuần này

Mỗi nhóm nộp một trang thiết kế trước khi triển khai sâu.

1 Data unit + partition key

Một đơn vị dữ liệu là gì? Chia theo trường nào?

2 Map / worker contract

Input, output, lỗi và tính idempotent

3 Aggregation / merge

Gom theo key hay merge top-k/result thế nào?

4 Bottleneck hypothesis

Dự đoán CPU, RAM, disk, network hay skew

5 Scalability experiment

≥ 3 data sizes $\times$ ≥ 2 resource configs

6 Evidence

raw log, config, metric, biểu đồ và error analysis

CHỐT CHƯƠNG 2

MapReduce = hợp đồng chia và gom dữ liệu. Hadoop = lưu trữ + điều phối + chịu lỗi. Giá trị nằm ở thiết kế và bằng chứng scale.